

A9.1. UT3 PracticaGuiada 1:TestsUnitarios

Memoria de Prácticas — Testing con JUnit en Apache NetBeans

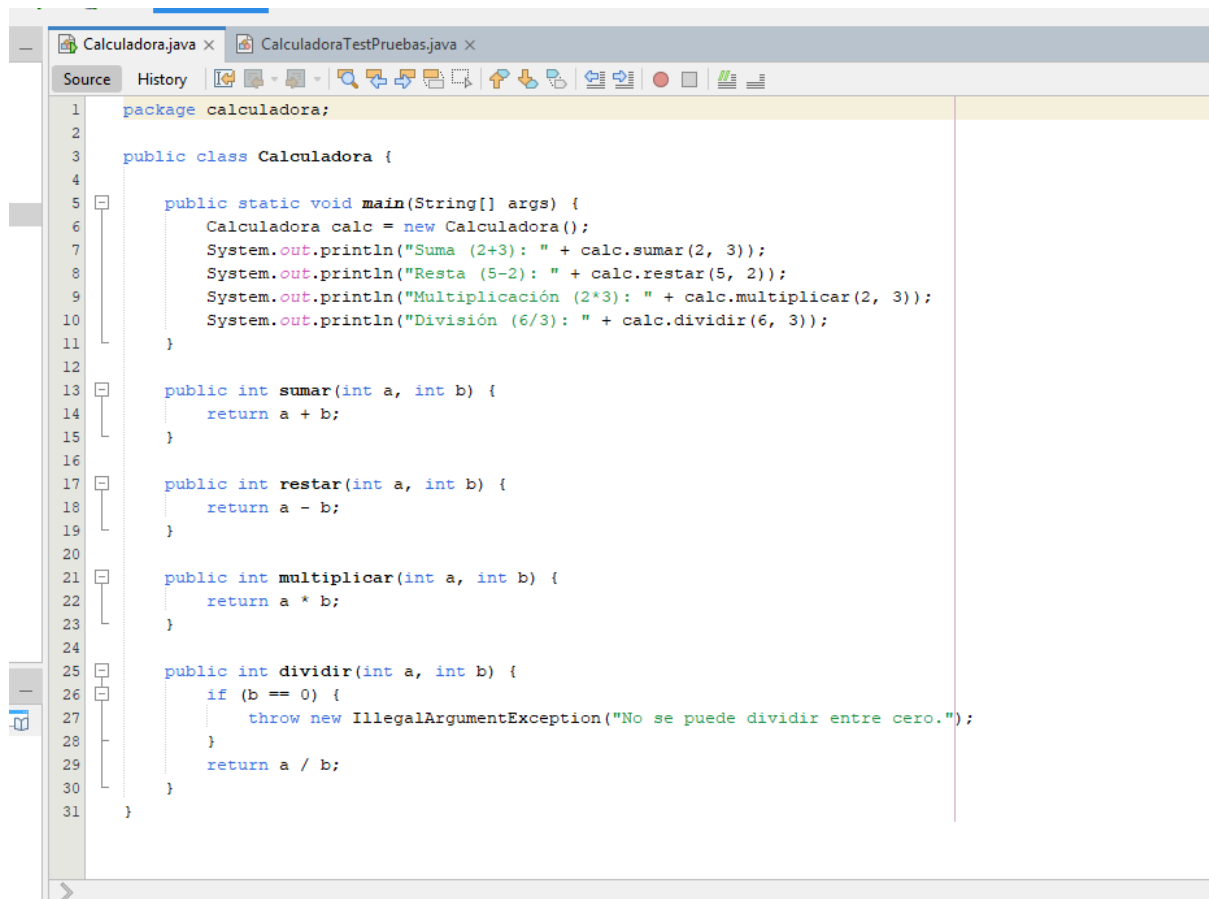
Práctica 1: Calculadora

Descripción

En esta práctica se ha desarrollado una clase `Calculadora` en Java que implementa las operaciones básicas de suma, resta, multiplicación y división. Además, se ha creado una clase de pruebas `CalculadoraTestPruebas` utilizando JUnit 4 para verificar el correcto funcionamiento de cada operación.

Clase principal — `Calculadora.java`

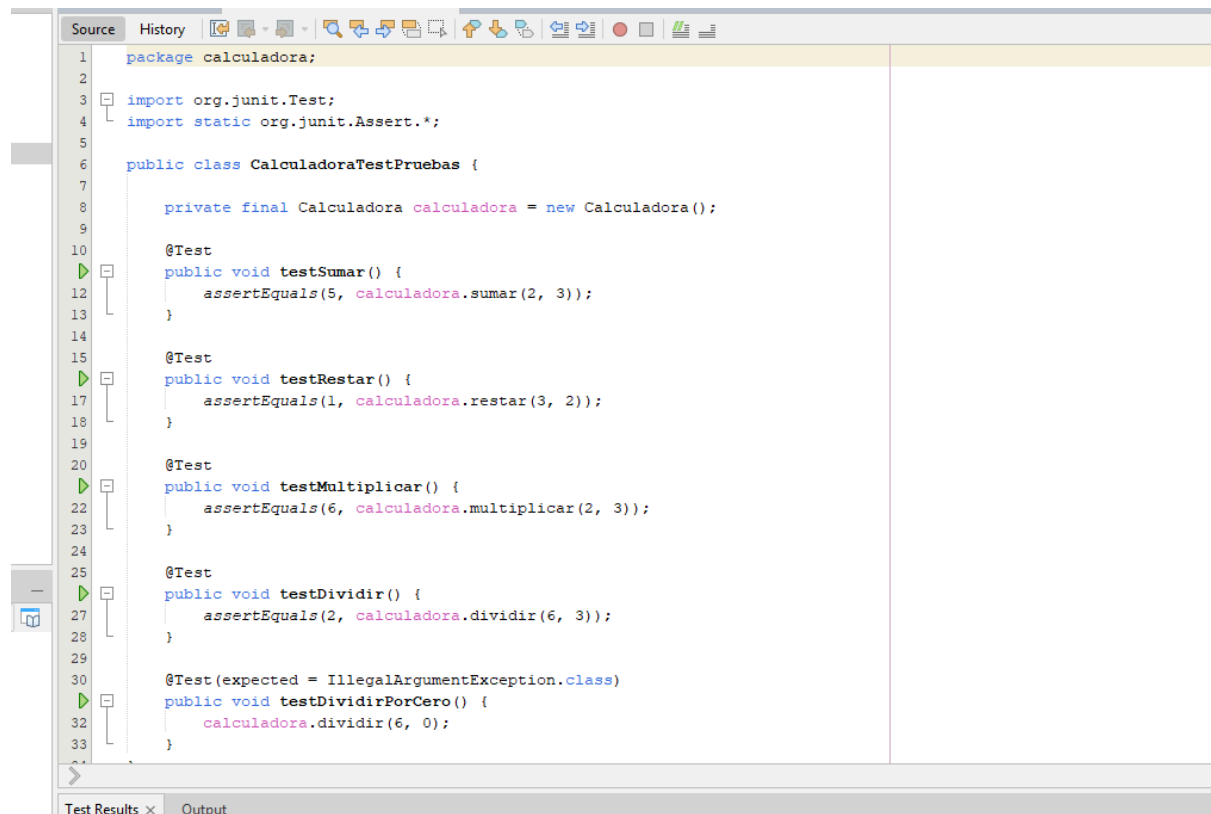
La clase `Calculadora` contiene los métodos `sumar`, `restar`, `multiplicar` y `dividir`. El método `dividir` incluye una validación para lanzar una excepción de tipo `IllegalArgumentException` cuando el divisor es cero.



```
1 package calculadora;
2
3 public class Calculadora {
4
5     public static void main(String[] args) {
6         Calculadora calc = new Calculadora();
7         System.out.println("Suma (2+3): " + calc.sumar(2, 3));
8         System.out.println("Resta (5-2): " + calc.restar(5, 2));
9         System.out.println("Multiplicación (2*3): " + calc.multiplicar(2, 3));
10        System.out.println("División (6/3): " + calc.dividir(6, 3));
11    }
12
13    public int sumar(int a, int b) {
14        return a + b;
15    }
16
17    public int restar(int a, int b) {
18        return a - b;
19    }
20
21    public int multiplicar(int a, int b) {
22        return a * b;
23    }
24
25    public int dividir(int a, int b) {
26        if (b == 0) {
27            throw new IllegalArgumentException("No se puede dividir entre cero.");
28        }
29        return a / b;
30    }
31 }
```

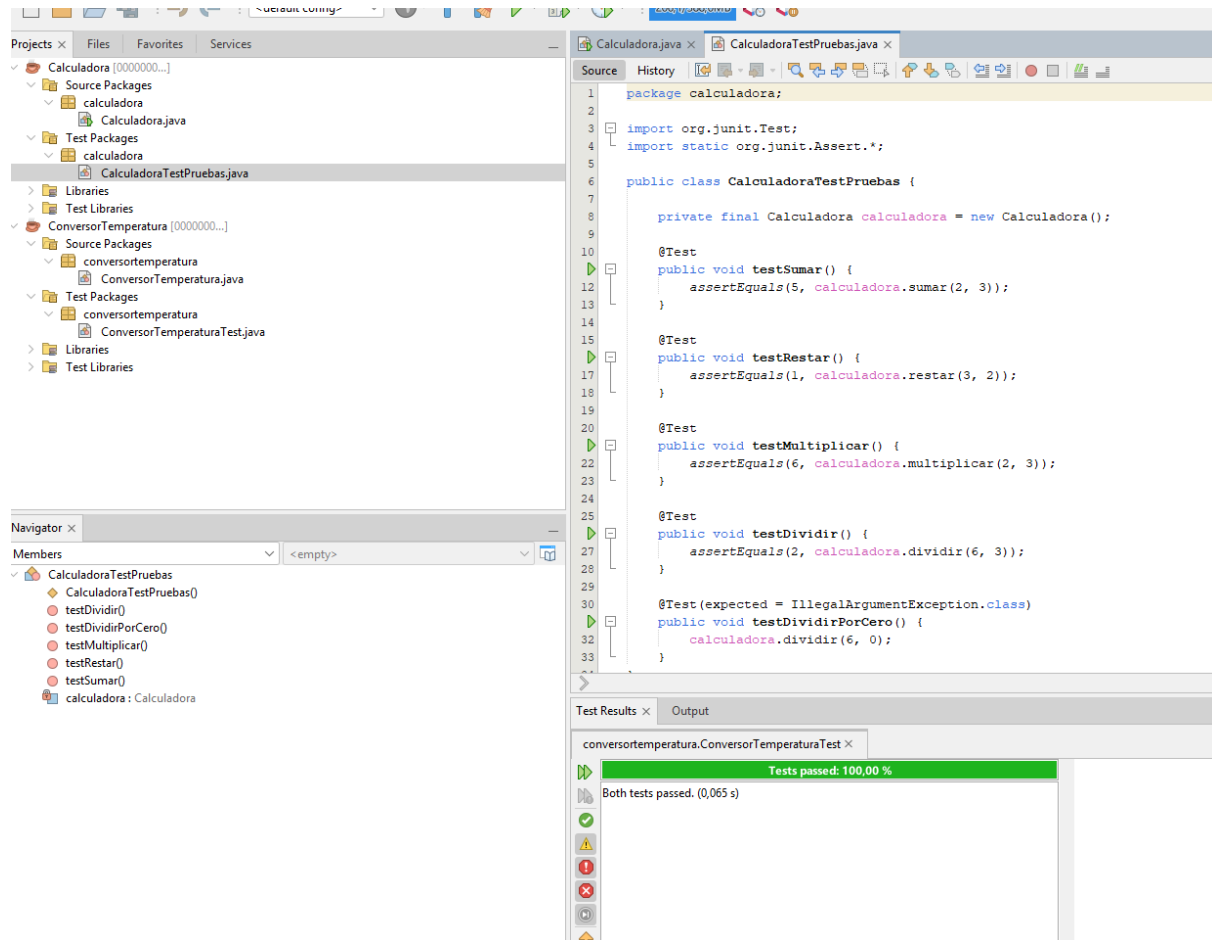
Clase de pruebas — CalculadoraTestPruebas.java

La clase de pruebas contiene cinco métodos de test que verifican cada operación de la calculadora, incluyendo un test específico para comprobar que la división entre cero lanza la excepción esperada.



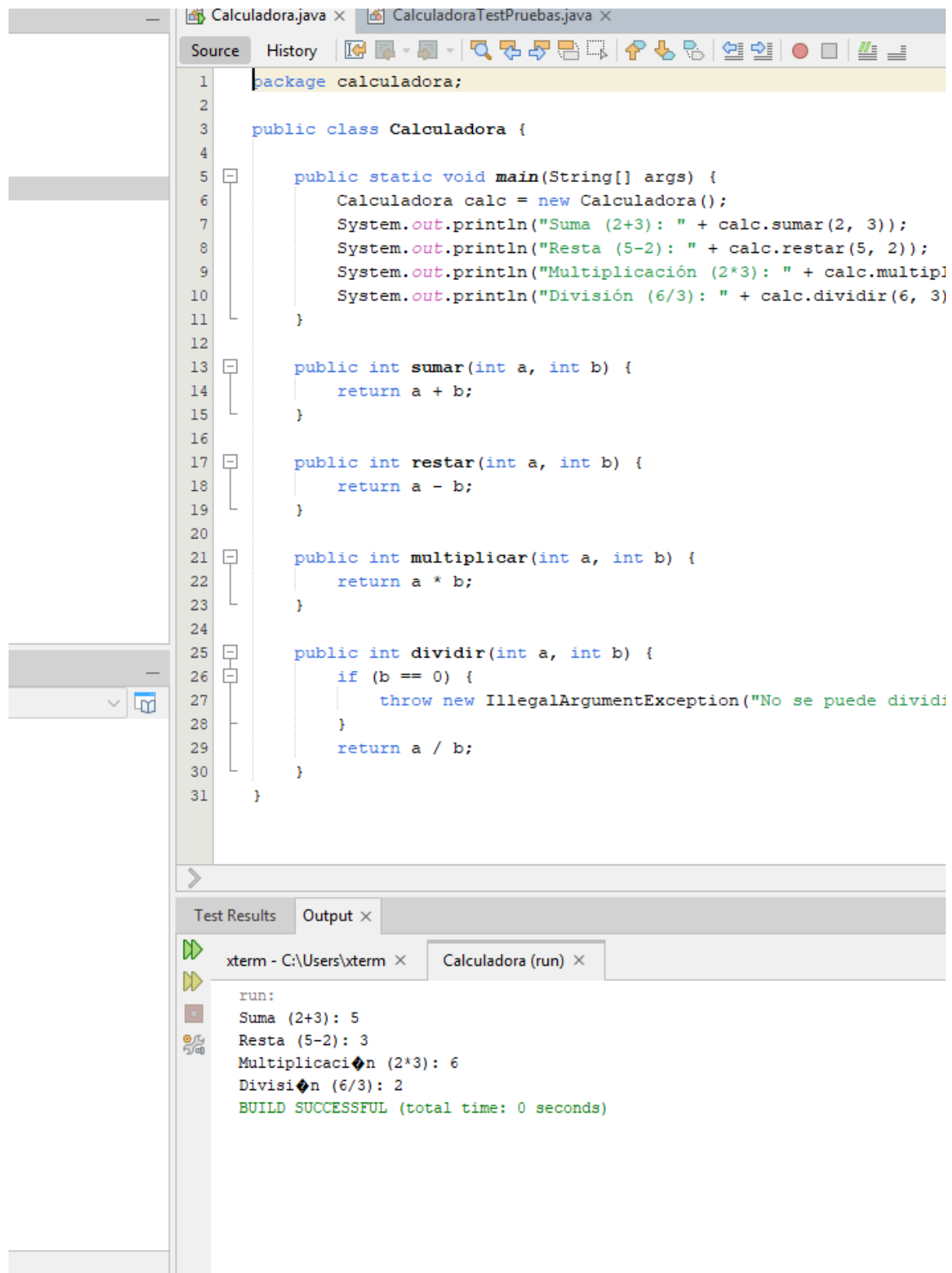
Ejecución de los tests

Se han ejecutado los cinco tests mediante la opción "Test File" de Apache NetBeans, obteniendo un resultado satisfactorio en todos ellos.



Ejecución del programa principal

Se ha ejecutado el método `main` de la clase `Calculadora` para verificar el funcionamiento del programa, obteniendo los resultados esperados en consola.



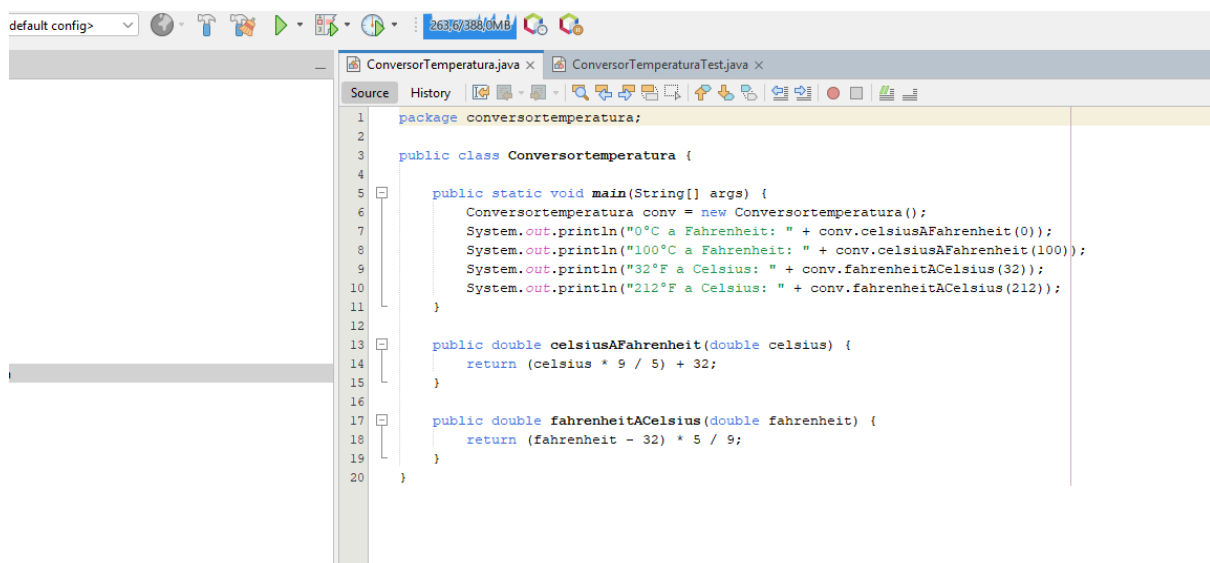
Práctica 2: Conversor de Temperatura

Descripción

En esta práctica se ha desarrollado una clase `Conversortemperatura` en Java que implementa la conversión entre grados Celsius y Fahrenheit en ambas direcciones. Se ha creado también una clase de pruebas `ConversorTemperaturaTest` utilizando JUnit 4 para verificar la corrección de las conversiones.

Clase principal — `Conversortemperatura.java`

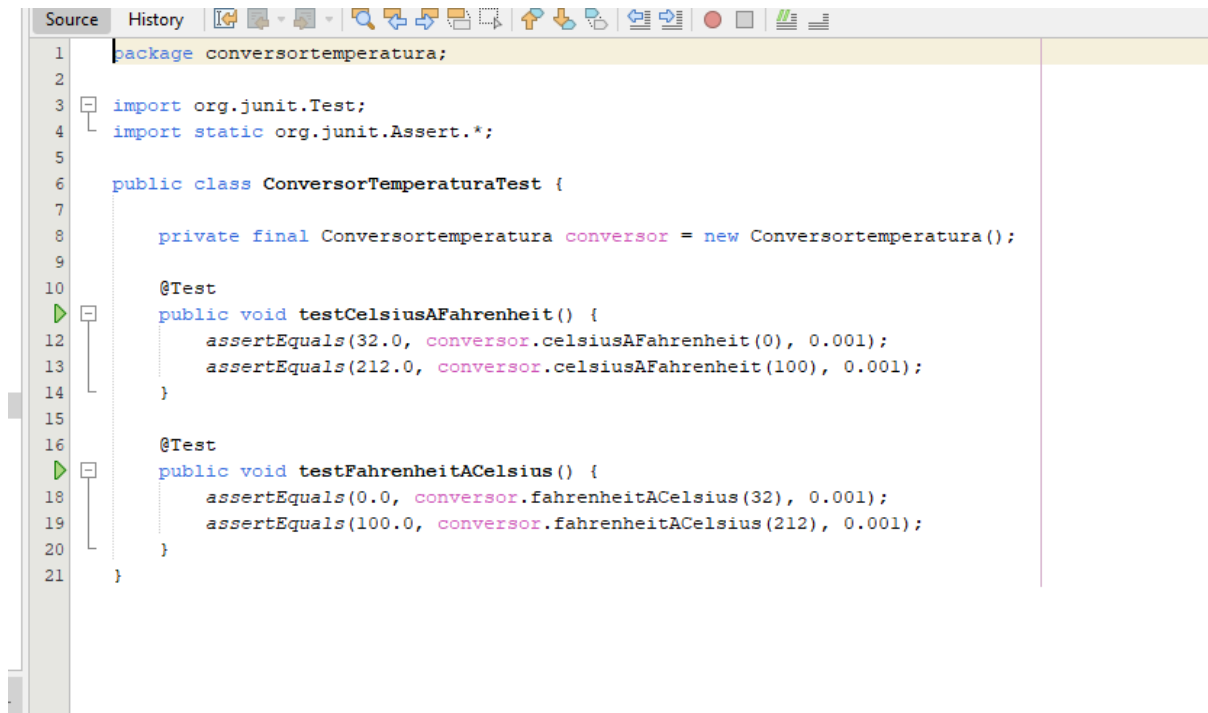
La clase `Conversortemperatura` contiene los métodos `celsiusAFahrenheit` y `fahrenheitACelsius`, que realizan las conversiones aplicando las fórmulas matemáticas correspondientes.

A screenshot of an IDE window showing the source code of the `Conversortemperatura.java` file. The window has a title bar with "default config" and a memory usage indicator "263/388.0MB". The code is as follows:

```
1 package conversortemperatura;
2
3 public class Conversortemperatura {
4
5     public static void main(String[] args) {
6         Conversortemperatura conv = new Conversortemperatura();
7         System.out.println("0°C a Fahrenheit: " + conv.celsiusAFahrenheit(0));
8         System.out.println("100°C a Fahrenheit: " + conv.celsiusAFahrenheit(100));
9         System.out.println("32°F a Celsius: " + conv.fahrenheitACelsius(32));
10        System.out.println("212°F a Celsius: " + conv.fahrenheitACelsius(212));
11    }
12
13    public double celsiusAFahrenheit(double celsius) {
14        return (celsius * 9 / 5) + 32;
15    }
16
17    public double fahrenheitACelsius(double fahrenheit) {
18        return (fahrenheit - 32) * 5 / 9;
19    }
20 }
```

Clase de pruebas — `ConversorTemperaturaTest.java`

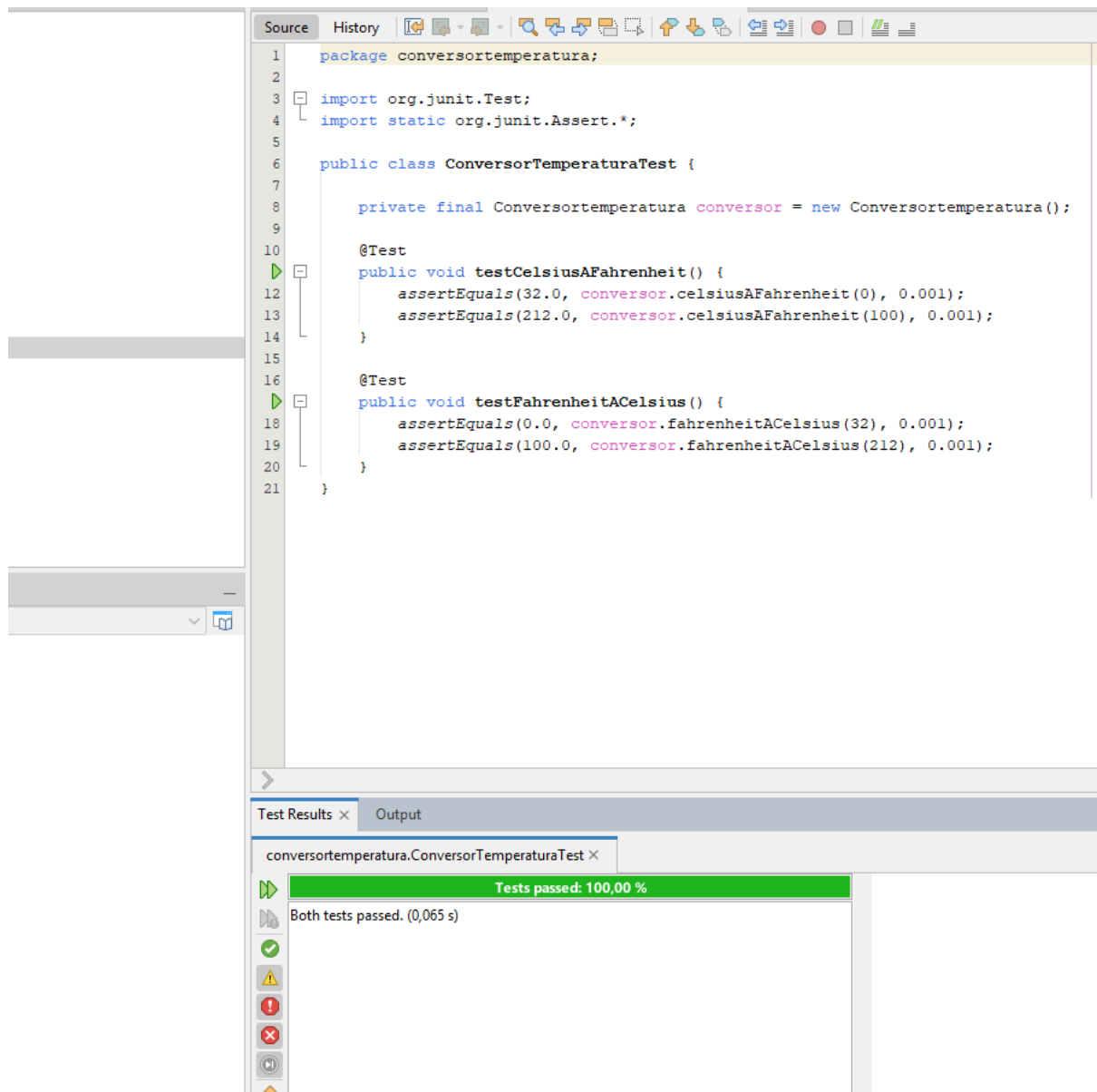
La clase de pruebas contiene dos métodos de test que verifican las dos conversiones utilizando valores conocidos como referencia, con una tolerancia de 0.001 para la comparación de valores decimales.

A screenshot of an IDE window showing a Java source file. The window has a title bar with 'Source' and 'History' tabs. The code is for a package named 'convertortemperatura'. It imports 'org.junit.Test' and 'org.junit.Assert.*'. A public class 'ConversorTemperaturaTest' is defined. Inside the class, there is a private final instance of 'Conversortemperatura' named 'conversor'. There are two test methods: 'testCelsiusAFahrenheit()' and 'testFahrenheitACelsius()'. Both methods use 'assertEquals' to verify the results of the conversion methods. The first test method checks the conversion of 0 and 100 Celsius to Fahrenheit. The second test method checks the conversion of 32 and 212 Fahrenheit to Celsius. The code is line-numbered from 1 to 21. The IDE interface includes a toolbar with various icons for file operations and a left margin with line numbers and foldable code blocks.

```
1 package convertortemperatura;
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class ConversorTemperaturaTest {
7
8     private final Conversortemperatura conversor = new Conversortemperatura();
9
10    @Test
11    public void testCelsiusAFahrenheit() {
12        assertEquals(32.0, conversor.celsiusAFahrenheit(0), 0.001);
13        assertEquals(212.0, conversor.celsiusAFahrenheit(100), 0.001);
14    }
15
16    @Test
17    public void testFahrenheitACelsius() {
18        assertEquals(0.0, conversor.fahrenheitACelsius(32), 0.001);
19        assertEquals(100.0, conversor.fahrenheitACelsius(212), 0.001);
20    }
21 }
```

Ejecución de los tests

Se han ejecutado los dos tests mediante la opción "Test File" de Apache NetBeans, obteniendo un resultado satisfactorio en ambos.



Ejecución del programa principal

Se ha ejecutado el método `main` de la clase `Conversortemperatura` para verificar el funcionamiento del programa, obteniendo los resultados esperados en consola.

The screenshot shows the NetBeans IDE with two tabs: `ConversorTemperatura.java` and `ConversorTemperaturaTest.java`. The `Source` tab is active, displaying the following Java code:

```
1 package conversortemperatura;
2
3 public class Conversortemperatura {
4
5     public static void main(String[] args) {
6         Conversortemperatura conv = new Conversortemperatura();
7         System.out.println("0°C a Fahrenheit: " + conv.celsiusAFahrenheit(0));
8         System.out.println("100°C a Fahrenheit: " + conv.celsiusAFahrenheit(100));
9         System.out.println("32°F a Celsius: " + conv.fahrenheitACelsius(32));
10        System.out.println("212°F a Celsius: " + conv.fahrenheitACelsius(212));
11    }
12
13    public double celsiusAFahrenheit(double celsius) {
14        return (celsius * 9 / 5) + 32;
15    }
16
17    public double fahrenheitACelsius(double fahrenheit) {
18        return (fahrenheit - 32) * 5 / 9;
19    }
20 }
```

Below the code editor, the `Output` tab is active, showing the execution results:

```
run:
0°C a Fahrenheit: 32.0
100°C a Fahrenheit: 212.0
32°F a Celsius: 0.0
212°F a Celsius: 100.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Conclusión

En ambas prácticas se ha comprobado el uso de pruebas unitarias con JUnit 4 en Apache NetBeans, verificando que los métodos implementados funcionan correctamente y que los casos límite, como la división entre cero, se gestionan adecuadamente.

